

Frontend Documentation

@Larissa Quirino @Denyse Ferreira

🎨 Cronograma do Frontend (Dashboard React)

Equipe: Larissa Quirino dos Santos & Denyse Panza Clemente Ferreira

Objetivo: Desenvolver uma interface web em React (JavaScript, mas pode escalar para TypeScript a depender da equipe) com Tailwind CSS que consuma a API do backend, exiba métricas em gráficos, liste logs e alertas, e ofereça uma experiência clara para o monitoramento do sistema.

 Estrutura do Cronograma

Semana	Período	Foco Principal
1	21/04 – 27/04	Setup do projeto e componentes mock
2	28/04 – 04/05	Consumo da API de métricas e gráficos básicos
3	05/05 – 11/05	Listagem de logs e refinamento do dashboard
4	12/05 – 18/05	Painel de alertas e melhorias visuais
5	19/05 – 25/05	Integração com detecção de ataques (alertas dinâmicos)
6	26/05 – 01/06	Ajustes finos e preparação para apresentação parcial
7	02/06 – 05/06	Polimento, responsividade e documentação

♦ Semana 1 | 21 – 27 abr

🎯 **Meta da Semana:** Projeto React criado, estrutura de pastas definida e componentes mock estáticos.

📋 Tarefas

- 1.1 Criar projeto React com Vite e TypeScript:

```
1 npm create vite@latest frontend -- --template react-ts
2
```

- 1.2 Instalar e configurar Tailwind CSS conforme a [documentação oficial](#).
- 1.3 Definir estrutura de pastas:

```
1 src/
2 |— features/
3 |   |— metrics/ # Tudo relacionado a métricas
4 |   |   |— components/
5 |   |   |   |— MetricsChart.jsx
6 |   |   |   |— MetricCard.jsx
7 |   |   |— services/
8 |   |   |   |— metricsApi.js # funções fetch para métricas
9 |   |   |— routes.jsx # se houver rotas específicas (ex: /metrics/:hostId)
10 |
11 |   |— logs/ # Tudo relacionado a logs
12 |   |   |— components/
13 |   |   |   |— LogsTable.jsx
14 |   |   |   |— LogFilter.jsx
15 |   |   |— services/
16 |   |   |   |— logsApi.js
17 |   |   |— routes.jsx
18 |
19 |   |— alerts/ # Tudo relacionado a alertas
20 |   |   |— components/
21 |   |   |   |— AlertsPanel.jsx
22 |   |   |   |— AlertBadge.jsx
23 |   |   |— services/
24 |   |   |   |— alertsApi.js
25 |   |   |— routes.jsx
26 |
27 |   |— hosts/ # Seleção e status de hosts
28 |   |   |— components/
29 |   |   |   |— HostSelector.jsx
30 |   |   |— services/
31 |   |   |   |— hostsApi.js
32 |   |   |— routes.jsx
33 |
34 |   |— dashboard/ # Visão geral / página principal
35 |   |   |— components/
36 |   |   |   |— DashboardLayout.jsx
37 |   |   |— routes.jsx
38 |
39 |— shared/ # Código reutilizável entre features
40 |   |— components/
41 |   |   |— Layout.jsx
42 |   |   |— Navbar.jsx
43 |   |   |— LoadingSpinner.jsx
44 |   |— hooks/
45 |   |   |— useApi.js # hook customizado para chamadas HTTP
46 |   |— utils/
47 |   |   |— formatters.js # formatar datas, bytes, etc.
48
```

```
49 | App.jsx
50 | main.jsx
51 | index.css (inclui Tailwind)
52
```

- **1.4** Criar tipos TypeScript baseados no contrato JSON acordado com Backend e Agente:
 - `Host`, `Metric`, `LogEntry`, `Alert`
- **1.5** Criar componentes mock (estáticos) para:
 - Card de métrica principal (CPU / RAM)
 - Tabela de logs (com dados fake)
 - Painel de alertas (com placeholder)
- **1.6** Configurar ESLint e Prettier para manter padronização de código.

✓ Entregáveis da Semana

- Projeto rodando localmente com Tailwind.
- Componentes visuais estáticos (mock) prontos para serem conectados à API.

♦ Semana 2 | 28 abr – 04 mai

🎯 **Meta da Semana:** Conectar o frontend à API real e exibir métricas em gráficos.

📋 Tarefas

- **2.1** Criar serviço `api.ts` com funções para consumir os endpoints:
 - `fetchMetrics(hostId: string, limit?: number): Promise<Metric[]>`
 - `fetchHosts(): Promise<Host[]>`
 - Utilizar `axios` ou `fetch` com tipagem adequada.
- **2.2** Configurar variáveis de ambiente para a URL da API (`VITE_API_URL`).
- **2.3** Instalar e configurar **Chart.js** e **react-chartjs-2**:

```
1 | npm install chart.js react-chartjs-2
2 |
```

- **2.4** Criar componente `MetricsChart` que exibe gráfico de linha para CPU e Memória ao longo do tempo.
- **2.5** Implementar seletor de host (dropdown) para filtrar métricas.
- **2.6** Substituir dados mock do Card por dados reais vindos da API (última métrica recebida).

✓ Entregáveis da Semana

- Dashboard exibindo gráficos reais de CPU/Memória.
- Seletor de host funcional.

- Comunicação com API estabelecida.
-

♦ Semana 3 | 05 – 11 mai

🎯 **Meta da Semana:** Exibir logs recebidos do agente e melhorar a experiência do dashboard.

📋 Tarefas

- **3.1** Adicionar função `fetchLogs(hostId: string, limit?: number)` no serviço `api.ts`.
- **3.2** Criar componente `LogsTable` que lista logs recentes com colunas:
 - Timestamp (formatado)
 - Tipo (auth, system, etc.)
 - Mensagem resumida (extraída do `raw_line` ou `parsed_data`)
 - IP de origem (se disponível)
- **3.3** Implementar paginação ou scroll infinito na tabela de logs (dependendo do volume).
- **3.4** Adicionar filtro por tipo de log (ex.: apenas `auth`).
- **3.5** Melhorar layout do dashboard: usar grid responsivo, cards organizados.
- **3.6** Adicionar indicador de status do host (online/offline) baseado na última métrica recebida.

✅ Entregáveis da Semana

- Tabela de logs funcional e integrada à API.
 - Dashboard com layout mais refinado e responsivo.
-

♦ Semana 4 | 12 – 18 mai

🎯 **Meta da Semana:** Implementar painel de alertas e visualização de eventos de segurança.

📋 Tarefas

- **4.1** Adicionar função `fetchAlerts(status?: string)` no serviço `api.ts`.
- **4.2** Criar componente `AlertsPanel` que exibe alertas ativos:
 - Badge de severidade (cores diferentes para `high`, `medium`, `low`).
 - Informações: tipo (`brute_force`, `port_scan`), IP de origem, timestamp.
 - Botão para marcar como resolvido (chamada `PATCH /api/alerts/<id>/resolve`).
- **4.3** Adicionar polling automático a cada 10-15 segundos para atualizar métricas, logs e alertas sem recarregar a página.

- **4.4** Criar uma área de notificação visual (toast ou banner) quando um novo alerta crítico surgir.
- **4.5** Testar a integração com alertas gerados pelo backend (simular ataque).

✅ Entregáveis da Semana

- Painel de alertas funcional.
 - Atualização em tempo real via polling.
 - Interface reativa a novos eventos de segurança.
-

♦ Semana 5 | 19 – 25 mai

🎯 **Meta da Semana:** Integração com detecção de Brute Force e ajustes de usabilidade.

📋 Tarefas

- **5.1** Garantir que os alertas de `brute_force` apareçam corretamente no painel.
- **5.2** Adicionar gráfico ou contador de tentativas de login falhas por IP (se a API fornecer esse agregado, ou calcular no frontend).
- **5.3** Melhorar a exibição de logs: destacar linhas com falhas de autenticação (ex.: fundo vermelho claro).
- **5.4** Implementar um modo escuro (dark mode) usando Tailwind (opcional, mas agrega valor visual).
- **5.5** Realizar testes de usabilidade com a equipe e coletar feedback.

✅ Entregáveis da Semana

- Dashboard com destaque visual para eventos de segurança.
 - Feedback da equipe incorporado.
-

♦ Semana 6 | 26 mai – 01 jun ⚠️ Apresentação Parcial 29/05

🎯 **Meta da Semana:** Deixar o frontend estável e apresentável para a avaliação parcial.

📋 Tarefas

- **6.1** Corrigir bugs visuais e de carregamento de dados.
- **6.2** Garantir que o dashboard funcione com os dados reais que estarão disponíveis na parcial.
- **6.3** Preparar uma demonstração guiada: o que clicar, o que mostrar.
- **6.4** Criar um estado de "carregando" (skeleton loaders) para melhorar a experiência durante as requisições.

- **6.5** Verificar compatibilidade com navegadores (Chrome e Firefox no mínimo).

✓ Entregáveis da Semana

- Frontend pronto para ser exibido na apresentação parcial.
 - Navegação fluida e sem erros aparentes.
-

♦ Semana 7 | 02 – 05 jun 🚀 MVP Deadline

🎯 **Meta da Semana:** Polimento final, responsividade mobile e documentação.

📋 Tarefas

- **7.1** Ajustar responsividade para telas menores (tablet e mobile) – pelo menos garantir que não quebre.
- **7.2** Implementar tratamento de erros de API (ex.: exibir mensagem amigável se o servidor estiver fora).
- **7.3** Adicionar um pequeno **footer** com informações do projeto (Universidade, nomes).
- **7.4** Criar um `README.md` do frontend com:
 - Como instalar dependências.
 - Como configurar variáveis de ambiente.
 - Como rodar em desenvolvimento e gerar build de produção.
- **7.5** Realizar build de produção e testar servindo via Nginx (dentro de container Docker, se for o caso).

✓ Entregáveis da Semana

- Frontend polido, responsivo e documentado.
 - Build de produção pronto para ser integrado ao Docker Compose geral.
-

🎤 Período de Pitch (06 – 12 jun)

Tarefas do Frontend para o Pitch

- **P.1** Garantir que o dashboard está visualmente atraente e com dados de demonstração.
 - **P.2** Preparar slides ou capturas de tela para incluir na apresentação geral.
 - **P.3** Estar disponível para explicar as decisões de UI/UX e a integração com o backend.
-

⚠ Observações Técnicas

1. CORS:

O backend deve estar configurado para aceitar requisições da origem do frontend (ex.: `http://localhost:5173`). Caso contrário, o navegador bloqueará as chamadas. Coordenem com Douglas e Fernando.

2. Variáveis de Ambiente:

Criem um arquivo `.env.example` com as variáveis necessárias (`VITE_API_URL`), mas **nunca** commitem o `.env` real com valores sensíveis.

3. Chart.js:

Para gráficos de série temporal, utilizem o eixo X como `time` e forneçam os dados no formato adequado. Exemplo:

```
1 {
2   labels: metrics.map(m => new Date(m.timestamp).toLocaleTimeString()),
3   datasets: [{
4     label: 'CPU %',
5     data: metrics.map(m => m.cpu_percent),
6     borderColor: 'rgb(75, 192, 192)',
7   }]
8 }
9
```

4. Polling vs WebSockets:

Para o MVP, o polling é suficiente e mais simples. Se quiserem um diferencial, podem mencionar que uma evolução seria usar WebSockets para atualização em tempo real.

5. Acessibilidade:

Utilizem cores com contraste adequado e labels nos botões. Isso conta pontos na apresentação.